to be careful when making such changes that the semantics of the operation are not changed in a problematic fashion: *if you don't hold the required locks for the entire duration of an operation, you're exposing yourself to race conditions*.

Sometimes, there just isn't an appropriate level of granularity because not all accesses to the data structure require the same level of protection. In this case, it might be appropriate to use an alternative mechanism, instead of a plain `std::mutex`.

3.3. Alternative facilities for protecting shared data

Although they're the most general mechanism, mutexes aren't the only game in town when it comes to protecting shared data; there are alternatives that provide more appropriate protection in specific scenarios.

One particularly extreme (but remarkably common) case is where the shared data needs protection only from concurrent access while it's being initialized, but after that no explicit synchronization is required. This might be because the data is read-only once created, and so there are no possible synchronization issues, or it might be because the necessary protection is performed implicitly as part of the operations on the data. In either case, locking a mutex after the data has been initialized, purely in order to protect the initialization, is unnecessary and a needless hit to performance. It's for this reason that the C++ Standard provides a mechanism purely for protecting shared data during initialization.

3.3.1. Protecting shared data during initialization

Suppose you have a shared resource that's so expensive to construct that you want to do so only if it's actually required; maybe it opens a database connection or allocates a lot of memory. *Lazy initialization* such as this is common in single-threaded code—each operation that requires the resource first checks to see if it has been initialized and then initializes it before use if not: